

# **FPGA-Based Simple Calculator**

With Addition, Subtraction, Multiplication, Division, and Squaring

Sheraz Javed

Wayne State University

ECE 2610: Digital Logic Lab Sec: 006

Instructor: Mustafa Alhababi

April 21, 2026

## Table of Contents

|                                            |       |
|--------------------------------------------|-------|
| Table of Contents .....                    | 2     |
| Abstract .....                             | 3     |
| Introduction .....                         | 4     |
| Design .....                               | 5-9   |
| System Overview and Module Hierarchy ..... | 5     |
| Input Mapping .....                        | 5     |
| Debouncer Module .....                     | 6     |
| Finite State Machine Module .....          | 6     |
| Arithmetic Logic Unit Module .....         | 8     |
| Seven-Segment Display Driver Module .....  | 8-9   |
| Constraints File .....                     | 9     |
| Source Code .....                          | 10-24 |
| Top Module: calculator.v .....             | 10-11 |
| Debouncer Module: debouncer.v .....        | 12    |
| FSM Module: calc_fsm.v .....               | 13-14 |
| ALU Module: calc_alu.v .....               | 15-16 |
| Display Driver Module: seg7_driver.v ..... | 17-19 |
| Testbench: calculator_tb.v .....           | 20-22 |
| Constraints: calculator.xdc .....          | 23-24 |
| Results .....                              | 25-31 |
| Simulation Results .....                   | 25    |
| Hardware Testing: Multiplication .....     | 26    |
| Hardware Testing: Addition .....           | 27    |
| Hardware Testing: Subtraction .....        | 28    |
| Hardware Testing: Division .....           | 29    |
| Hardware Testing: Squaring .....           | 30    |
| Summary of Test Results .....              | 31    |
| Conclusion .....                           | 32-33 |

## **Abstract**

This project focused on designing and building a simple calculator using a Nexys A7-100T FPGA board. The calculator takes number inputs from 1 to 5 using the board's push buttons and performs five operations: addition, multiplication, subtraction, division, and squaring. The system was built using a modular Verilog design with four main parts. These include a button debouncer, a finite state machine controller, an arithmetic logic unit, and a seven-segment display driver. A top-level module connects all these parts. Subtraction was done using two's complement, and an LED was used to show negative results. A constraints file was used to map signals to the correct pins on the board. A testbench was also created to check the design before using the hardware. Testing on the board showed that all operations worked correctly. This project helped us better understand digital logic design, finite state machines, and how FPGA systems work.

## **Introduction**

Digital systems are the base of almost every electronic device we use today, from simple calculators to advanced processors. Learning how these systems are designed at the hardware level is an important part of digital logic. In this project, we designed and built a working calculator on an FPGA board to apply these ideas in a real system.

The calculator uses the Nexys A7-100T development board. This board includes five push buttons, sixteen slide switches, eight seven-segment display digits, sixteen LEDs, and a 100 MHz clock. These features were used to control input, operations, and output. The calculator takes number inputs from 1 to 5. Each push button represents one number. The user presses a button to enter the first value, then presses another button for the second value. A slide switch is used to choose the operation.

The operations include addition, multiplication, subtraction, division, and squaring. The result is shown on the seven-segment display. If the subtraction result is negative, an LED turns on to show the sign. The design uses a modular structure. The Verilog code is divided into four main parts, which are a debouncer, a finite state machine, an arithmetic logic unit, and a display driver. A top-level module connects all these parts. An XDC file was used to connect each input and output signal to the correct pin on the board. A testbench was also created to simulate the system in Vivado before loading it onto the FPGA. This helped verify that the state machine and calculations worked correctly. The next sections explain the design in more detail, show the results, and discuss what was learned from the project.

## **Design**

### **System Overview and Module Hierarchy**

The calculator design uses a hierarchical modular architecture. The top-level module, named "calculator," contains no logic of its own. Instead, it instantiates four sub-modules and wires them together using internal signals. The sub-modules are:

1. debouncer: Synchronizes and debounces the five raw push button inputs, producing clean one-clock-cycle pulses.
2. calc\_fsm: The finite state machine that manages the calculator's flow, capturing operands, selecting operations, and triggering computation.
3. calc\_alu: The arithmetic logic unit that performs the selected operation (add, multiply, subtract, divide, or square) when triggered by the FSM.
4. seg7\_driver: The seven-segment display driver that multiplexes the four active digits on the Nexys A7 display, showing operands and results.

The top-level module also contains the operation switch rising-edge detector, which monitors SW1 through SW5 for transitions from low to high, and the reset wire derived from SW0. All external I/O ports (clock, buttons, switches, segments, anodes, LED) are defined in the top module and passed down to the appropriate sub-modules.

### **Input Mapping**

The five push buttons are mapped as follows: BTNL (left) enters digit 1, BTNR (right) enters digit 2, BTND (down) enters digit 3, BTNU (up) enters digit 4, and BTNC (center) enters digit 5. Pressing any button automatically locks in that digit and advances the FSM to the next state, eliminating the need for a separate confirm button.

The slide switches control operations and reset. SW0 is the master reset: flipping it up returns the FSM to idle and clears all stored values. SW1 selects addition, SW2 selects multiplication, SW3 selects subtraction, SW4 selects division, and SW5 selects squaring. The top module detects the rising edge of each switch so that the operation triggers once when the switch is flipped, not continuously while it is held.

## **Debouncer Module**

Mechanical push buttons do not give a clean signal when pressed and instead create rapid changes called bounce. This can make the FPGA think the button was pressed multiple times. The debouncer module fixes this by using a 20 bit counter that runs at 100 MHz and overflows about every 10 milliseconds. The button signals first go through a two stage synchronizer to make them stable, then they are only checked when the counter overflows. A rising edge detector compares the current state to the previous one and creates a single one clock cycle pulse for each press. This means that no matter how messy the button contact is, the FSM only sees one clean pulse for each button press.

Finite State Machine Module

The `calc_fsm` module controls the calculator's flow using five states:

**S0 (Idle):** All registers are cleared. The display is blank. The FSM waits for the user to press a digit button.

**S1 (Got First Operand):** The first operand (FO) has been captured. The FSM now waits for either a second digit press or the square switch.

**S2 (Got Second Operand):** The second operand (SO) has been captured. The FSM waits for the user to flip one of the operation switches.

**S3 (Compute):** This state lasts exactly one clock cycle. The FSM asserts a "compute" pulse that triggers the ALU, then immediately transitions to Display.

**S4 (Display):** The result is held on the display. The FSM remains here until the user resets with SW0.

For squaring, the FSM transitions directly from S1 to S3, bypassing S2, since only one operand is needed. The FSM also exposes its current state to the display driver so that the driver knows which digits to show at each point in the flow.

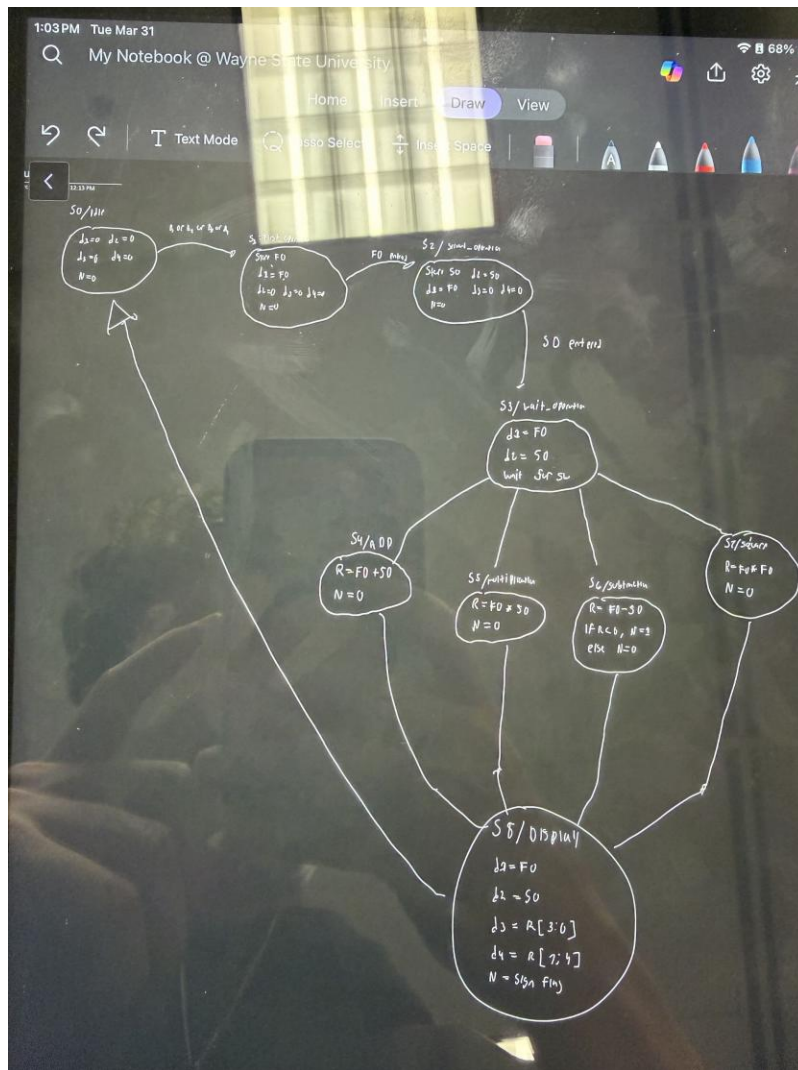


Figure 1: Finite State Machine Diagram

## **Arithmetic Logic Unit Module**

The `calc_alu` module performs the selected arithmetic operation when it receives a one-cycle compute pulse from the FSM. The ALU supports five operations:

Addition computes  $FO + SO$  by zero-extending both three-bit operands to eight bits and adding them. The maximum result is  $5 + 5 = 10$ .

Multiplication computes  $FO$  times  $SO$  directly. The maximum result is 5 times 5 = 25, which fits in eight bits.

Subtraction uses explicit two's complement arithmetic. The second operand  $SO$  is first zero-extended to eight bits. Then all bits are inverted to form the one's complement. Then one is added to form the two's complement. The two's complement of  $SO$  is added to the zero-extended  $FO$ . If the most significant bit of the eight-bit result is set, the result is negative: the ALU takes the two's complement of the result to obtain the absolute value for display and asserts the negative flag. If the MSB is clear, the result is positive or zero and is stored directly.

Division computes the integer quotient  $FO$  divided by  $SO$ . Since inputs range from 1 to 5, division by zero cannot occur under normal use, but the design includes a guard that outputs an error sentinel if  $SO$  is somehow zero.

Squaring computes  $FO$  times  $FO$ . The maximum result is 5 times 5 = 25.

## **Seven-Segment Display Driver Module**

The `seg7_driver` module handles the multiplexed seven-segment display on the Nexys A7 board. The board shares a single cathode bus across all eight digits, so only one digit can be physically lit at any instant. The driver cycles through the four active digits fast enough that persistence of vision makes them all appear lit simultaneously.

An 18-bit counter selects which of the four active digits ( $AN0$  through  $AN3$ ) is currently enabled. At 100 MHz, this cycles through all four digits at approximately 381 Hz each, well above the flicker threshold.  $AN4$  through  $AN7$  are permanently driven high (off).

For each digit position, a combinational block selects what value to show:  $AN3$  displays the first operand,  $AN2$  displays the second operand (blank for square operations),  $AN1$  displays either the tens digit of the result, a minus sign if the result is negative, or is blanked if the result is a single



digit, and AN0 displays the ones digit of the result. The selected four-bit value is fed into a seven-segment decoder that outputs the correct active-low cathode pattern for digits 0 through 9, plus a minus-sign pattern.

## **Constraints File**

The XDC constraints file maps every port in the top-level module to its physical pin on the Nexys A7-100T board. The clock input is mapped to pin E3 with a 10 ns period constraint. The five push buttons are mapped to their documented pins (P17, M17, P18, M18, N17). The six switches are mapped to pins J15, L16, M13, R15, R17, and T18. The seven segment cathodes are mapped to pins T10, R10, K16, K13, P15, T11, and L18. The decimal point is on pin H15. The eight anodes are mapped to J17, J18, T9, J14, P14, T14, K2, and U13. The negative LED is on pin H17.

## Source Code

### Top Module: calculator.v

The top-level module instantiates and connects all four sub-modules. It contains no computational logic of its own, only port declarations, internal wires, the operation switch edge detector, and module instantiations.

```
`timescale 1ns / 1ps

// =====
// Module: calculator (Top Module)
// Board: Nexys A7-100T (xc7a100tcs9324-1)
//
// Top-level module that instantiates and connects:
// 1. debouncer - cleans push button inputs
// 2. calc_fsm - finite state machine controller
// 3. calc_alu - arithmetic logic unit
// 4. seg7_driver - multiplexed seven-segment display driver
//
// Buttons (active-high):
// BTNL=1, BTNR=2, BTND=3, BTNU=4, BTNC=5
//
// Switches:
// SW0=Reset, SW1=Add, SW2=Multiply, SW3=Subtract,
// SW4=Divide, SW5=Square
//
// Outputs:
// seg[6:0] = shared 7-seg cathodes (active-low, gfedcba)
// dp = decimal point (always off)
// an[7:0] = anodes (active-low)
// led_n = LED[0], negative indicator
// =====

module calculator (
    input clk, // 100 MHz system clock
    input [4:0] btn, // {BTNC, BTNU, BTND, BTNR, BTNL}
    input [5:0] sw, // {SW5, SW4, SW3, SW2, SW1, SW0}
    output [6:0] seg, // 7-seg cathodes
    output dp, // decimal point
    output [7:0] an, // 7-seg anodes
    output led_n // negative LED
);

// — Reset —
wire rst = sw[0];

// — Internal wires —
wire [4:0] btn_pulse; // debounced button pulses
wire [2:0] FO, SO; // operands from FSM
wire [2:0] op_select; // operation from FSM
wire compute; // compute trigger from FSM
wire [2:0] state; // current state from FSM
wire [7:0] result; // result from ALU
wire neg_flag; // negative flag from ALU

// — Operation switch rising-edge detector —
reg [4:0] sw_op_prev;
wire [4:0] sw_op = sw[5:1];
wire [4:0] sw_op_rise;
```

```
always @(posedge clk) begin
    if (rst)
        sw_op_prev <= 0;
    else
        sw_op_prev <= sw_op;
end
assign sw_op_rise = sw_op & ~sw_op_prev;

// — Sub-module instantiations —

// 1. Button Debouncer
debouncer u_debouncer (
    .clk      (clk),
    .rst      (rst),
    .btn_raw  (btn),
    .btn_pulse (btn_pulse)
);

// 2. Finite State Machine
calc_fsm u_fsm (
    .clk      (clk),
    .rst      (rst),
    .btn_pulse (btn_pulse),
    .sw_op_rise (sw_op_rise),
    .FO       (FO),
    .SO       (SO),
    .op_select (op_select),
    .compute   (compute),
    .state     (state)
);

// 3. Arithmetic Logic Unit
calc_alu u_alu (
    .clk      (clk),
    .rst      (rst),
    .compute   (compute),
    .FO       (FO),
    .SO       (SO),
    .op_select (op_select),
    .result    (result),
    .neg_flag  (neg_flag)
);

// 4. Seven-Segment Display Driver
seg7_driver u_display (
    .clk      (clk),
    .rst      (rst),
    .state     (state),
    .FO       (FO),
    .SO       (SO),
    .op_select (op_select),
    .result    (result),
    .neg_flag  (neg_flag),
    .seg       (seg),
    .dp        (dp),
    .an        (an)
);

// — Negative LED —
assign led_n = neg_flag;

endmodule
```

## Debouncer Module: debouncer.v

Synchronizes and debounces the five push button inputs using a two-stage synchronizer and a 10 MS sampling counter.

```
`timescale 1ns / 1ps

// =====
// Module: debouncer
// Debounces 5 push buttons with ~10ms sampling window.
// Outputs a one-clock-cycle pulse for each rising edge detected.
// =====

module debouncer (
    input      clk,
    input      rst,
    input [4:0] btn_raw,      // raw button inputs
    output [4:0] btn_pulse    // debounced one-shot pulses
);

    // Double-flop synchronizer
    reg [4:0] sync1, sync2;

    // Debounce sampling
    reg [19:0] debounce_cnt; // 2^20 @ 100MHz ≈ 10.5ms
    wire debounce_tick = (debounce_cnt == 0);
    reg [4:0] btn_stable;
    reg [4:0] btn_pulse_reg;

    always @(posedge clk) begin
        if (rst) begin
            sync1      <= 0;
            sync2      <= 0;
            btn_stable  <= 0;
            btn_pulse_reg <= 0;
            debounce_cnt <= 0;
        end else begin
            // Synchronize inputs
            sync1 <= btn_raw;
            sync2 <= sync1;

            // Free-running counter
            debounce_cnt <= debounce_cnt + 1;

            // Default: no pulse
            btn_pulse_reg <= 0;

            if (debounce_tick) begin
                // Rising-edge detect: was 0, now 1
                btn_pulse_reg <= sync2 & ~btn_stable;
                btn_stable    <= sync2;
            end
        end
    end

    assign btn_pulse = btn_pulse_reg;

endmodule
```

**FSM Module: calc\_fsm.v**

Controls the calculator flow through five states: Idle, Got FO, Got SO, Compute, and Display.

```
`timescale 1ns / 1ps

// =====
// Module: calc_fsm
// Finite State Machine for the calculator.
// Manages states: IDLE → GOT_FO → GOT_SO → COMPUTE → DISPLAY
// Outputs operands, operation select, and a compute trigger.
// =====

module calc_fsm (
    input        clk,
    input        rst,
    input  [4:0] btn_pulse,    // debounced button pulses
    input  [4:0] sw_op_rise,    // rising edge of operation switches
    output reg [2:0] FO,        // first operand (1-5)
    output reg [2:0] SO,        // second operand (1-5)
    output reg [2:0] op_select, // selected operation
    output reg      compute,    // one-cycle pulse to trigger ALU
    output reg [2:0] state      // current state (exposed for display logic)
);

    // State encoding
    localparam S_IDLE     = 3'd0;
    localparam S_GOT_FO   = 3'd1;
    localparam S_GOT_SO   = 3'd2;
    localparam S_COMPUTE  = 3'd3;
    localparam S_DISPLAY  = 3'd4;

    // Operation encoding
    localparam OP_NONE = 3'd0;
    localparam OP_ADD  = 3'd1;
    localparam OP_MUL  = 3'd2;
    localparam OP_SUB  = 3'd3;
    localparam OP_DIV  = 3'd4;
    localparam OP_SQ   = 3'd5;

    // Priority encoder: which digit was pressed?
    reg [2:0] pressed_digit;
    reg      any_btn;

    always @(*) begin
        pressed_digit = 0;
        any_btn = 0;
        if (btn_pulse[0]) begin pressed_digit = 3'd1; any_btn = 1; end
        else if (btn_pulse[1]) begin pressed_digit = 3'd2; any_btn = 1; end
        else if (btn_pulse[2]) begin pressed_digit = 3'd3; any_btn = 1; end
        else if (btn_pulse[3]) begin pressed_digit = 3'd4; any_btn = 1; end
        else if (btn_pulse[4]) begin pressed_digit = 3'd5; any_btn = 1; end
    end

    // State machine
    always @(posedge clk) begin
        if (rst) begin
            state      <= S_IDLE;
            FO         <= 0;
            SO         <= 0;
            op_select  <= OP_NONE;
            compute    <= 0;
        end else begin
            compute <= 0; // default
        end
    end
endmodule
```

```

case (state)
  S_IDLE: begin
    op_select <= OP_NONE;
    SO      <= 0;
    if (any_btn) begin
      FO      <= pressed_digit;
      state <= S_GOT_FO;
    end
  end

  S_GOT_FO: begin
    if (sw_op_rise[4]) begin          // SW5 = square
      op_select <= OP_SQ;
      compute  <= 1;
      state    <= S_COMPUTE;
    end
    else if (any_btn) begin
      SO      <= pressed_digit;
      state <= S_GOT_SO;
    end
  end

  S_GOT_SO: begin
    if (sw_op_rise[0]) begin          // SW1 = add
      op_select <= OP_ADD;
      compute  <= 1;
      state    <= S_COMPUTE;
    end
    else if (sw_op_rise[1]) begin     // SW2 = multiply
      op_select <= OP_MUL;
      compute  <= 1;
      state    <= S_COMPUTE;
    end
    else if (sw_op_rise[2]) begin     // SW3 = subtract
      op_select <= OP_SUB;
      compute  <= 1;
      state    <= S_COMPUTE;
    end
    else if (sw_op_rise[3]) begin     // SW4 = divide
      op_select <= OP_DIV;
      compute  <= 1;
      state    <= S_COMPUTE;
    end
    else if (sw_op_rise[4]) begin     // SW5 = square
      op_select <= OP_SQ;
      compute  <= 1;
      state    <= S_COMPUTE;
    end
  end

  S_COMPUTE: begin
    // One cycle only, move to display
    state <= S_DISPLAY;
  end

  S_DISPLAY: begin
    // Hold until reset
  end

  default: state <= S_IDLE;
endcase
end
end
endmodule

```

**ALU Module: calc\_alu.v**

Performs addition, multiplication, subtraction (two's complement), division, and squaring when triggered.

```
`timescale 1ns / 1ps

// =====
// Module: calc_alu
// Arithmetic Logic Unit for the calculator.
// Performs: Add, Multiply, Subtract (2's complement), Divide, Square
// =====

module calc_alu (
    input      clk,
    input      rst,
    input      compute,      // one-cycle trigger from FSM
    input [2:0] FO,           // first operand
    input [2:0] SO,           // second operand
    input [2:0] op_select,    // operation code
    output reg [7:0] result,   // 8-bit result
    output reg      neg_flag  // asserted if subtraction result is negative
);

    // Operation encoding (must match calc_fsm)
    localparam OP_ADD  = 3'd1;
    localparam OP_MUL  = 3'd2;
    localparam OP_SUB  = 3'd3;
    localparam OP_DIV  = 3'd4;
    localparam OP_SQ   = 3'd5;

    always @(posedge clk) begin
        if (rst) begin
            result <= 0;
            neg_flag <= 0;
        end else if (compute) begin
            case (op_select)
                OP_ADD: begin
                    result <= {5'b0, FO} + {5'b0, SO};
                    neg_flag <= 0;
                end

                OP_MUL: begin
                    result <= FO * SO;
                    neg_flag <= 0;
                end

                OP_SUB: begin
                    // 2's complement subtraction: R = FO + (~SO + 1)
                    // Step 1: invert SO bits (1's complement)
                    // Step 2: add 1 (2's complement)
                    // Step 3: add to FO
                    // Step 4: check MSB for sign
                    begin : twos_comp_block
                        reg [7:0] so_inv;
                        reg [7:0] so_twos;
                        reg [7:0] sub_result;

                        so_inv      = ~{5'b0, SO};           // 1's complement
                        so_twos     = so_inv + 8'd1;          // 2's complement
                        sub_result = {5'b0, FO} + so_twos;    // FO + 2's comp of SO
                    end
                end
            endcase
        end
    end
endmodule
```

```
        if (sub_result[7]) begin
            // MSB set = negative result
            // Take 2's complement of result for magnitude
            result <= ~sub_result + 8'd1;
            neg_flag <= 1;
        end else begin
            // Positive or zero
            result <= sub_result;
            neg_flag <= 0;
        end
    end
end

OP_DIV: begin
    if (SO != 0)
        result <= {5'b0, FO} / {5'b0, SO};
    else
        result <= 8'hEE; // error sentinel
        neg_flag <= 0;
    end
end

OP_SQ: begin
    result <= FO * FO;
    neg_flag <= 0;
end

default: begin
    result <= 0;
    neg_flag <= 0;
end
endcase
end
end
endmodule
```



**Display Driver Module: seg7\_driver.v**

Multiplexes four digits on the seven-segment display at 381 Hz per digit and decodes BCD values to active-low cathode patterns.

```
`timescale 1ns / 1ps

// =====
// Module: seg7_driver
// Seven-segment multiplexing display driver for Nexys A7.
// Drives 4 of the 8 digits (AN0-AN3), keeps AN4-AN7 off.
//   AN3 = first operand (FO)
//   AN2 = second operand (SO), blank for square
//   AN1 = result tens digit or minus sign
//   AN0 = result ones digit
// =====

module seg7_driver (
    input      clk,
    input      rst,
    input [2:0] state,          // current FSM state
    input [2:0] FO,             // first operand
    input [2:0] SO,             // second operand
    input [2:0] op_select,      // operation (to detect square)
    input [7:0] result,         // ALU result
    input      neg_flag,        // negative indicator
    output [6:0] seg,           // 7-seg cathodes (active-low, gfedcba)
    output      dp,             // decimal point (always off)
    output [7:0] an             // anodes (active-low)
);

// State encoding (must match calc_fsm)
localparam S_IDLE      = 3'd0;
localparam S_GOT_FO    = 3'd1;
localparam S_GOT_SO    = 3'd2;
localparam S_COMPUTE   = 3'd3;
localparam S_DISPLAY   = 3'd4;

// Operation encoding
localparam OP_SQ = 3'd5;

// — Refresh counter —
// 100MHz / 2^18 ≈ 381 Hz per digit
reg [17:0] refresh_cnt;
wire [1:0] digit_sel = refresh_cnt[17:16];

always @(posedge clk) begin
    if (rst)
        refresh_cnt <= 0;
    else
        refresh_cnt <= refresh_cnt + 1;
end

// — BCD extraction of result —
wire [3:0] res_tens = result / 10;
wire [3:0] res_ones = result % 10;

// — Digit selection logic —
reg [3:0] d_val;
reg      show_minus;
reg [7:0] an_reg;
```

```

always @(*) begin
    d_val      = 4'd0;
    show_minus = 1'b0;
    an_reg     = 8'b11111111; // all off by default

    case (digit_sel)
        // AN0: result ones digit
        2'b00: begin
            if (state == S_DISPLAY) begin
                d_val = res_ones;
                an_reg = 8'b11111110;
            end
        end

        // AN1: result tens OR minus sign
        2'b01: begin
            if (state == S_DISPLAY) begin
                if (neg_flag) begin
                    show_minus = 1'b1;
                    an_reg     = 8'b11111101;
                end else if (res_tens != 0) begin
                    d_val = res_tens;
                    an_reg = 8'b11111101;
                end
            end
        end

        // AN2: second operand (blank for square)
        2'b10: begin
            if ((state == S_GOT_SO || state == S_COMPUTE || state == S_DISPLAY)
                && op_select != OP_SQ) begin
                d_val = {1'b0, SO};
                an_reg = 8'b11111011;
            end
        end

        // AN3: first operand (blank in IDLE)
        2'b11: begin
            if (state != S_IDLE) begin
                d_val = {1'b0, FO};
                an_reg = 8'b11110111;
            end
        end
    endcase
end

assign an = an_reg;
assign dp = 1'b1; // decimal point always off

// — Seven-segment decoder (active-low, common anode) —
// seg[6:0] = gfedcba
reg [6:0] seg_reg;

always @(*) begin
    if (show_minus) begin
        seg_reg = 7'b01111111; // minus sign: segment g on
    end else begin
        case (d_val)
            4'd0: seg_reg = 7'b1000000;
            4'd1: seg_reg = 7'b1111001;
            4'd2: seg_reg = 7'b0100100;
            4'd3: seg_reg = 7'b0110000;
            4'd4: seg_reg = 7'b0011001;
            4'd5: seg_reg = 7'b0010010;
            4'd6: seg_reg = 7'b0000010;
            4'd7: seg_reg = 7'b1111000;
            4'd8: seg_reg = 7'b0000000;
        endcase
    end
end

```

### *FPGA-Based Simple Calculator*

```
        4'd9: seg_reg = 7'b0010000;  
        default: seg_reg = 7'b1111111; // blank  
    endcase  
end  
end  
  
    assign seg = seg_reg;  
  
endmodule
```

**Testbench: calculator\_tb.v**

Exercises all five operations with various operand combinations using task-based helper functions.

```
`timescale 1ns / 1ps

module calculator_tb;

    // -----
    // DUT signals
    // -----
    reg      clk;
    reg [4:0] btn;
    reg [5:0] sw;
    wire [6:0] seg;
    wire      dp;
    wire [7:0] an;
    wire      led_n;

    // -----
    // Instantiate DUT
    // -----
    calculator uut (
        .clk      (clk),
        .btn      (btn),
        .sw       (sw),
        .seg      (seg),
        .dp       (dp),
        .an       (an),
        .led_n    (led_n)
    );

    // -----
    // 100 MHz clock
    // -----
    initial clk = 0;
    always #5 clk = ~clk;

    // -----
    // Helper task: press a button for ~15ms then release
    // The debouncer needs ~10ms, so 15ms guarantees detection.
    // In simulation with debounce_cnt[19:0], full cycle = 2^20 * 10ns ≈ 10.5ms
    //
    // FOR FASTER SIMULATION: In calculator.v, temporarily change:
    //   reg [19:0] debounce_cnt -> reg [3:0] debounce_cnt
    //   debounce_tick = (debounce_cnt == 0) (stays same)
    //   digit_sel = refresh_cnt[17:16] -> refresh_cnt[3:2]
    //   reg [17:0] refresh_cnt -> reg [3:0] refresh_cnt
    //
    // Then these delays can be much shorter (e.g., 500ns instead of 15_000_000ns)
    // RESTORE original values before synthesis!
    // -----

    // Use short delays assuming you've made the simulation speedup changes above.
    // If NOT, change 500 -> 15_000_000 and 300 -> 5_000_000 everywhere below.
    parameter PRESS_TIME    = 500; // button held time (ns)
    parameter RELEASE_TIME  = 300; // gap between actions (ns)
    parameter SW_WAIT       = 500; // wait after flipping switch (ns)

    task press_btn;
        input [2:0] which; // 0=BTNL(1), 1=BTNR(2), 2=BTND(3), 3=BTNU(4), 4=BTNC(5)
        begin
            #PRESS_TIME btn[which] = 1;
            #RELEASE_TIME btn[which] = 0;
            #SW_WAIT sw[which] = 1;
        end
    endtask
endmodule
```

```

        btn[which] = 1'b1;
        #PRESS_TIME;
        btn[which] = 1'b0;
        #RELEASE_TIME;
    end
endtask

task flip_sw;
    input [2:0] which; // 1=add, 2=mul, 3=sub, 4=div, 5=sq
    begin
        sw[which] = 1'b1;
        #SW_WAIT;
    end
endtask

task do_reset;
    begin
        sw[0] = 1'b1; // SW0 = reset
        #RELEASE_TIME;
        sw = 6'b0; // all switches off
        #RELEASE_TIME;
    end
endtask

// -----
// Test sequence
// -----
initial begin
    $dumpfile("calculator_tb.vcd");
    $dumpvars(0, calculator_tb);

    btn = 5'b0;
    sw = 6'b0;

    // Start with reset
    do_reset;

    // ==== TEST 1: 3 + 2 = 5 ====
    $display("--- TEST 1: 3 + 2 (Add) ---");
    press_btn(2); // BTND = digit 3 -> FO=3
    press_btn(1); // BTNR = digit 2 -> SO=2
    flip_sw(1); // SW1 = Add
    #RELEASE_TIME;
    $display(" led_n=%b (expect 0), seg=%b, an=%b", led_n, seg, an);
    do_reset;

    // ==== TEST 2: 4 * 5 = 20 ====
    $display("--- TEST 2: 4 * 5 (Multiply) ---");
    press_btn(3); // BTNU = digit 4 -> FO=4
    press_btn(4); // BTNC = digit 5 -> SO=5
    flip_sw(2); // SW2 = Multiply
    #RELEASE_TIME;
    $display(" led_n=%b (expect 0)", led_n);
    do_reset;

    // ==== TEST 3: 2 - 5 = -3 (negative) ====
    $display("--- TEST 3: 2 - 5 (Subtract, negative) ---");
    press_btn(1); // BTNR = digit 2 -> FO=2
    press_btn(4); // BTNC = digit 5 -> SO=5
    flip_sw(3); // SW3 = Subtract
    #RELEASE_TIME;
    $display(" led_n=%b (expect 1)", led_n);
    do_reset;

    // ==== TEST 4: 5 - 3 = 2 (positive) ====
    $display("--- TEST 4: 5 - 3 (Subtract, positive) ---");

```

## FPGA-Based Simple Calculator

```
press_btn(4);    // BTNC = digit 5 -> FO=5
press_btn(2);    // BTND = digit 3 -> SO=3
flip_sw(3);      // SW3 = Subtract
#RELEASE_TIME;
$display(" led_n=%b (expect 0)", led_n);
do_reset;

// ==== TEST 5: 4 / 2 = 2 ====
$display("--- TEST 5: 4 / 2 (Divide) ---");
press_btn(3);    // BTNU = digit 4 -> FO=4
press_btn(1);    // BTNR = digit 2 -> SO=2
flip_sw(4);      // SW4 = Divide
#RELEASE_TIME;
$display(" led_n=%b (expect 0)", led_n);
do_reset;

// ==== TEST 6: 5 squared = 25 ====
$display("--- TEST 6: 5 squared ---");
press_btn(4);    // BTNC = digit 5 -> FO=5
flip_sw(5);      // SW5 = Square (skips SO)
#RELEASE_TIME;
$display(" led_n=%b (expect 0)", led_n);
do_reset;

// ==== TEST 7: 3 squared = 9 ====
$display("--- TEST 7: 3 squared ---");
press_btn(2);    // BTND = digit 3 -> FO=3
flip_sw(5);      // SW5 = Square
#RELEASE_TIME;
$display(" led_n=%b (expect 0)", led_n);
do_reset;

// ==== TEST 8: 1 + 1 = 2 ====
$display("--- TEST 8: 1 + 1 (Add) ---");
press_btn(0);    // BTNL = digit 1 -> FO=1
press_btn(0);    // BTNL = digit 1 -> SO=1
flip_sw(1);      // SW1 = Add
#RELEASE_TIME;
$display(" led_n=%b (expect 0)", led_n);
do_reset;

// ==== TEST 9: 3 - 3 = 0 ====
$display("--- TEST 9: 3 - 3 (Subtract, zero) ---");
press_btn(2);    // BTND = digit 3 -> FO=3
press_btn(2);    // BTND = digit 3 -> SO=3
flip_sw(3);      // SW3 = Subtract
#RELEASE_TIME;
$display(" led_n=%b (expect 0)", led_n);
do_reset;

// ==== TEST 10: 5 / 3 = 1 (integer truncation) ====
$display("--- TEST 10: 5 / 3 (Divide, truncation) ---");
press_btn(4);    // BTNC = digit 5 -> FO=5
press_btn(2);    // BTND = digit 3 -> SO=3
flip_sw(4);      // SW4 = Divide
#RELEASE_TIME;
$display(" led_n=%b (expect 0)", led_n);
do_reset;

$display("=====");
$display("All tests complete. Check waveforms for details.");
$display("=====");
#100;
$finish;
end

endmodule
```

## Constraints: calculator.xdc

Maps all ports to Nexys A7-100T physical pins.

```
## =====
## Constraints: calculator.xdc
## Board: Nexys A7-100T (xc7a100tcsg324-1)
## =====

## --- Clock (100 MHz) ---
set_property -dict { PACKAGE_PIN E3  IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

## --- Push Buttons (active-high on Nexys A7) ---
## btn[0] = BTNL = digit 1
set_property -dict { PACKAGE_PIN P17 IOSTANDARD LVCMOS33 } [get_ports {btn[0]}]
## btn[1] = BTNR = digit 2
set_property -dict { PACKAGE_PIN M17 IOSTANDARD LVCMOS33 } [get_ports {btn[1]}]
## btn[2] = BTND = digit 3
set_property -dict { PACKAGE_PIN P18 IOSTANDARD LVCMOS33 } [get_ports {btn[2]}]
## btn[3] = BTNU = digit 4
set_property -dict { PACKAGE_PIN M18 IOSTANDARD LVCMOS33 } [get_ports {btn[3]}]
## btn[4] = BTNC = digit 5
set_property -dict { PACKAGE_PIN N17 IOSTANDARD LVCMOS33 } [get_ports {btn[4]}]

## --- Switches ---
## sw[0] = SW0 = Reset
set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 } [get_ports {sw[0]}]
## sw[1] = SW1 = Add
set_property -dict { PACKAGE_PIN L16 IOSTANDARD LVCMOS33 } [get_ports {sw[1]}]
## sw[2] = SW2 = Multiply
set_property -dict { PACKAGE_PIN M13 IOSTANDARD LVCMOS33 } [get_ports {sw[2]}]
## sw[3] = SW3 = Subtract
set_property -dict { PACKAGE_PIN R15 IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]
## sw[4] = SW4 = Divide
set_property -dict { PACKAGE_PIN R17 IOSTANDARD LVCMOS33 } [get_ports {sw[4]}]
## sw[5] = SW5 = Square
set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports {sw[5]}]

## --- Seven Segment Cathodes (active-low, shared bus) ---
## seg[6:0] = gfedcba
set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports {seg[0]}]
set_property -dict { PACKAGE_PIN R10 IOSTANDARD LVCMOS33 } [get_ports {seg[1]}]
set_property -dict { PACKAGE_PIN K16 IOSTANDARD LVCMOS33 } [get_ports {seg[2]}]
set_property -dict { PACKAGE_PIN K13 IOSTANDARD LVCMOS33 } [get_ports {seg[3]}]
set_property -dict { PACKAGE_PIN P15 IOSTANDARD LVCMOS33 } [get_ports {seg[4]}]
set_property -dict { PACKAGE_PIN T11 IOSTANDARD LVCMOS33 } [get_ports {seg[5]}]
set_property -dict { PACKAGE_PIN L18 IOSTANDARD LVCMOS33 } [get_ports {seg[6]}]

## --- Decimal Point (active-low) ---
set_property -dict { PACKAGE_PIN H15 IOSTANDARD LVCMOS33 } [get_ports dp]

## --- Seven Segment Anodes (active-low) ---
set_property -dict { PACKAGE_PIN J17 IOSTANDARD LVCMOS33 } [get_ports {an[0]}]
set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports {an[1]}]
set_property -dict { PACKAGE_PIN T9  IOSTANDARD LVCMOS33 } [get_ports {an[2]}]
set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports {an[3]}]
set_property -dict { PACKAGE_PIN P14 IOSTANDARD LVCMOS33 } [get_ports {an[4]}]
set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports {an[5]}]
set_property -dict { PACKAGE_PIN K2  IOSTANDARD LVCMOS33 } [get_ports {an[6]}]
```

## *FPGA-Based Simple Calculator*

```
set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports {an[7]}]

## — Negative LED (LED[0]) —————
set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports led_n]
```



## Results

### Simulation Results

The testbench was run in Vivado's built-in simulator (xsim). The chart shows all input and output signals over time. Because of the debounce and refresh counters sized for real hardware, the simulation shows the FSM in its initial idle state.

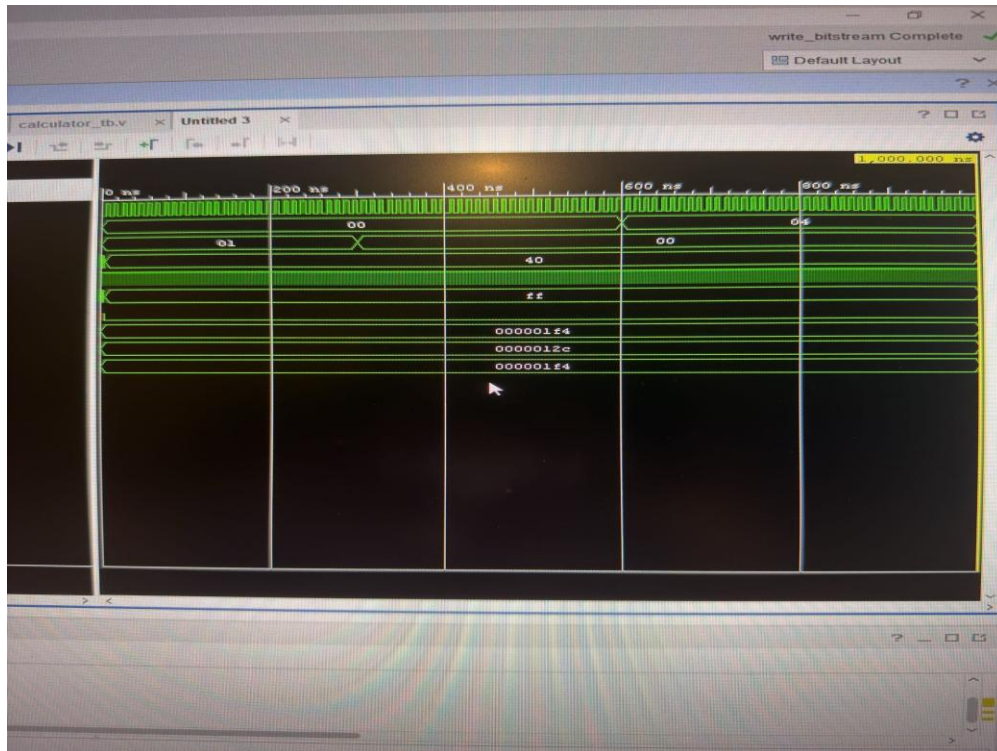


Figure 2: Testbench Simulation Waveform in Vivado

## Hardware Testing: Multiplication

The multiplication operation was tested by entering 5 as the first operand and 3 as the second operand, then flipping SW2. The display shows 5, 3, 15, reading as: first operand = 5, second operand = 3, result = 15. This is correct since 5 times 3 equals 15.

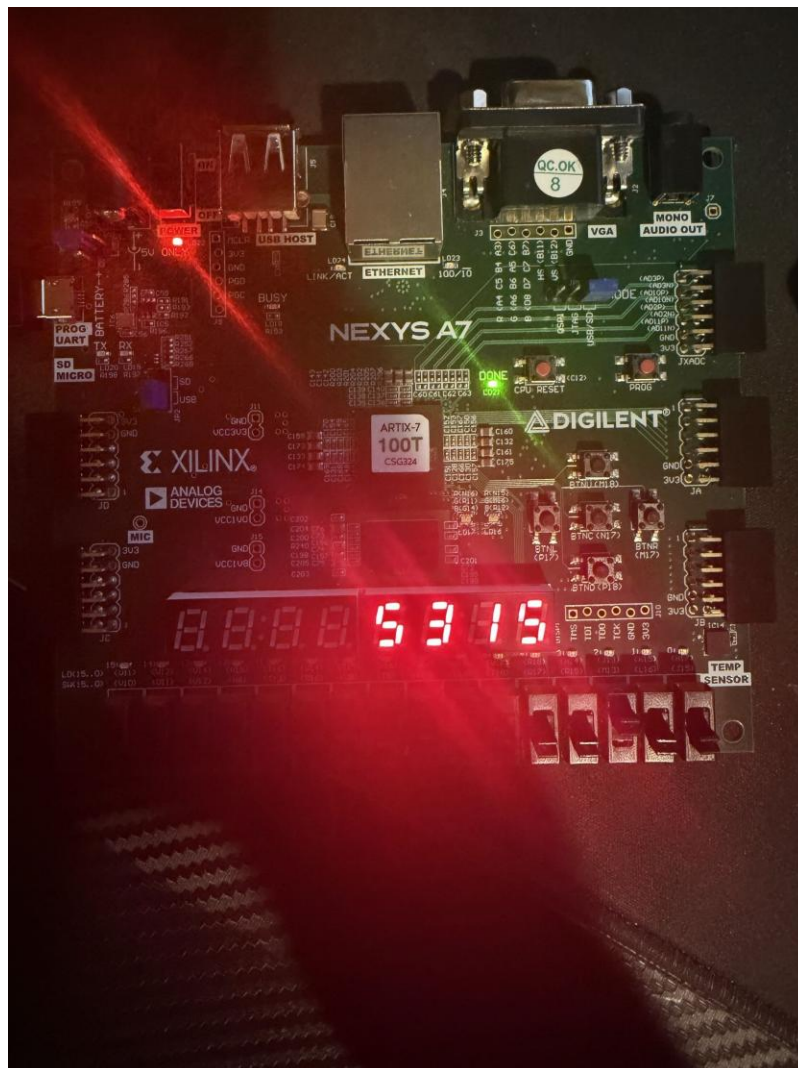


Figure 3: Multiplication Test ( $5 \times 3 = 15$ )

## Hardware Testing: Addition

The addition operation was tested by entering 5 as the first operand and 4 as the second operand, then flipping SW1. The display shows 5, 4, 9, reading as: first operand = 5, second operand = 4, result = 9. This is correct since 5 plus 4 equals 9.

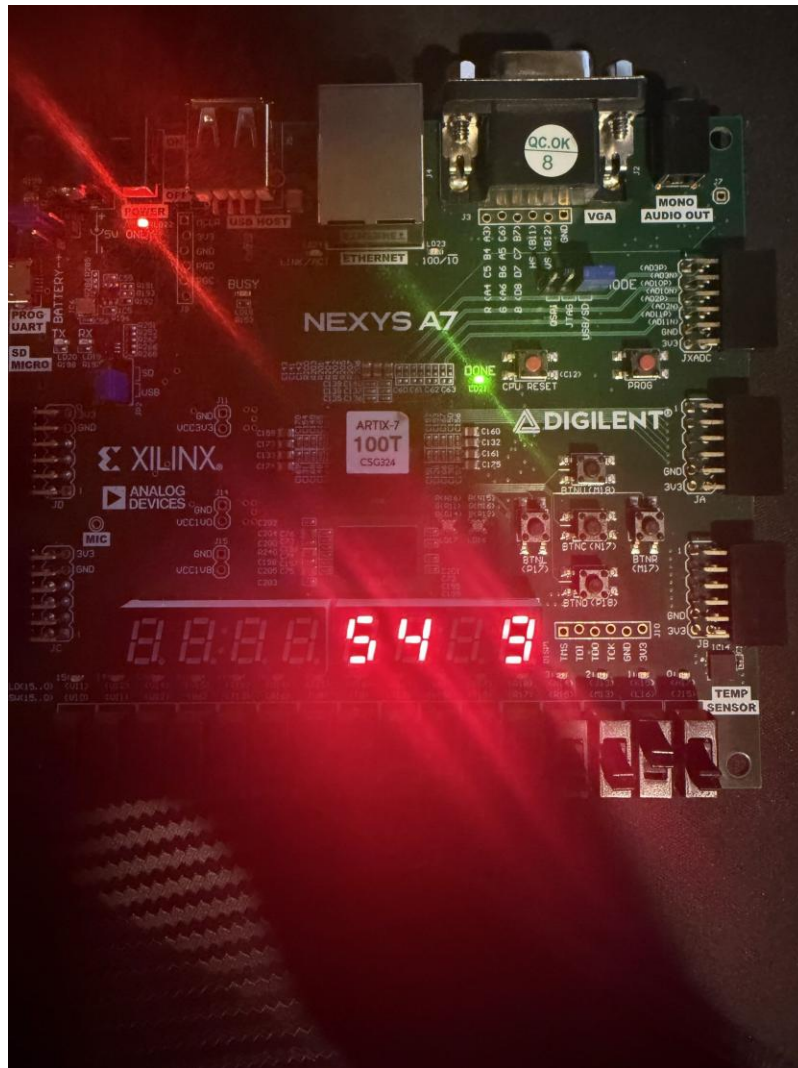


Figure 4: Addition Test ( $5 + 4 = 9$ )

## Hardware Testing: Subtraction

The subtraction operation was tested by entering 1 as the first operand and 4 as the second operand, then flipping SW3 (subtract). The display shows 1, 4, -3 and the negative LED is lit. This reads as: first operand = 1, second operand = 4, result = -3. This is correct since 1 minus 4 equals -3 using two's complement.

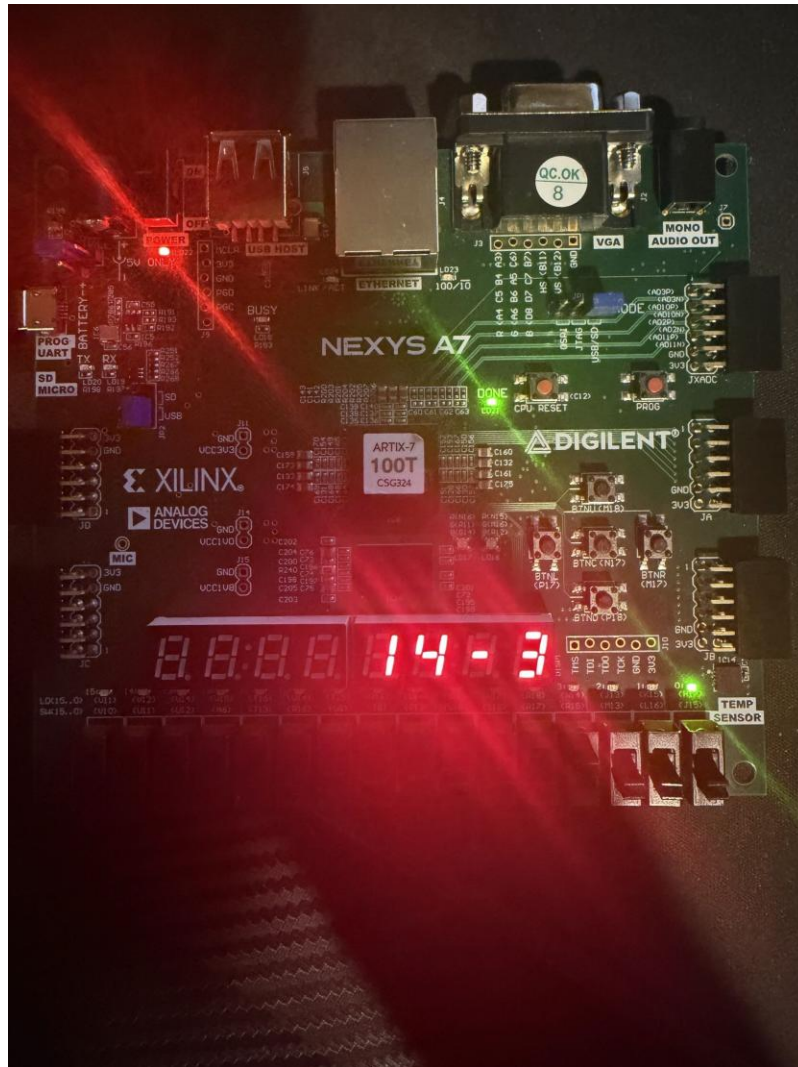


Figure 5: Subtraction Test ( $1 - 4 = -3$ , LED lit)



## Hardware Testing: Division

The division operation was tested by entering 4 as the first operand and 2 as the second operand, then flipping SW4. The display shows 4, 2, 2, reading as: first operand = 4, second operand = 2, result = 2. This is correct since 4 divided by 2 equals 2.

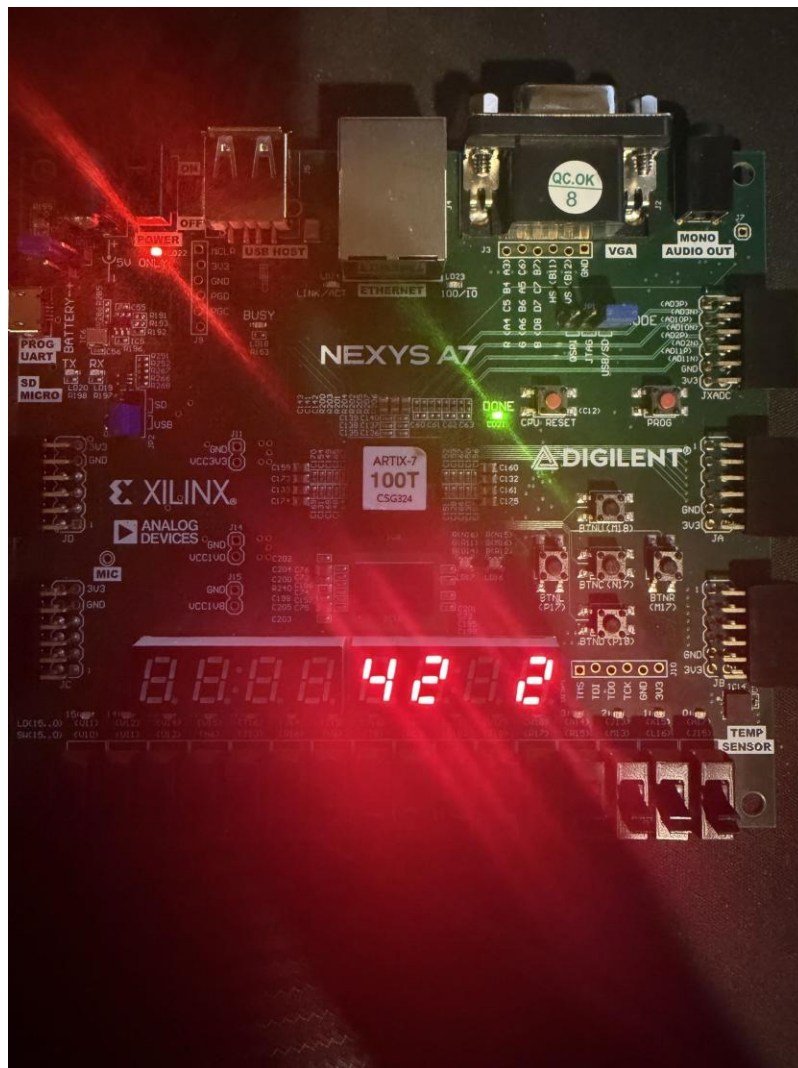


Figure 6: Division Test ( $4 / 2 = 2$ )

## Hardware Testing: Squaring

The squaring operation was tested by entering 5 as the first operand, then flipping SW5 without entering a second operand. The display shows 5, 25, reading as: first operand = 5, result = 25. The second operand position is blank since squaring only requires one operand. This is correct since 5 squared equals 25. The FSM correctly skipped the second operand and went directly to compute.

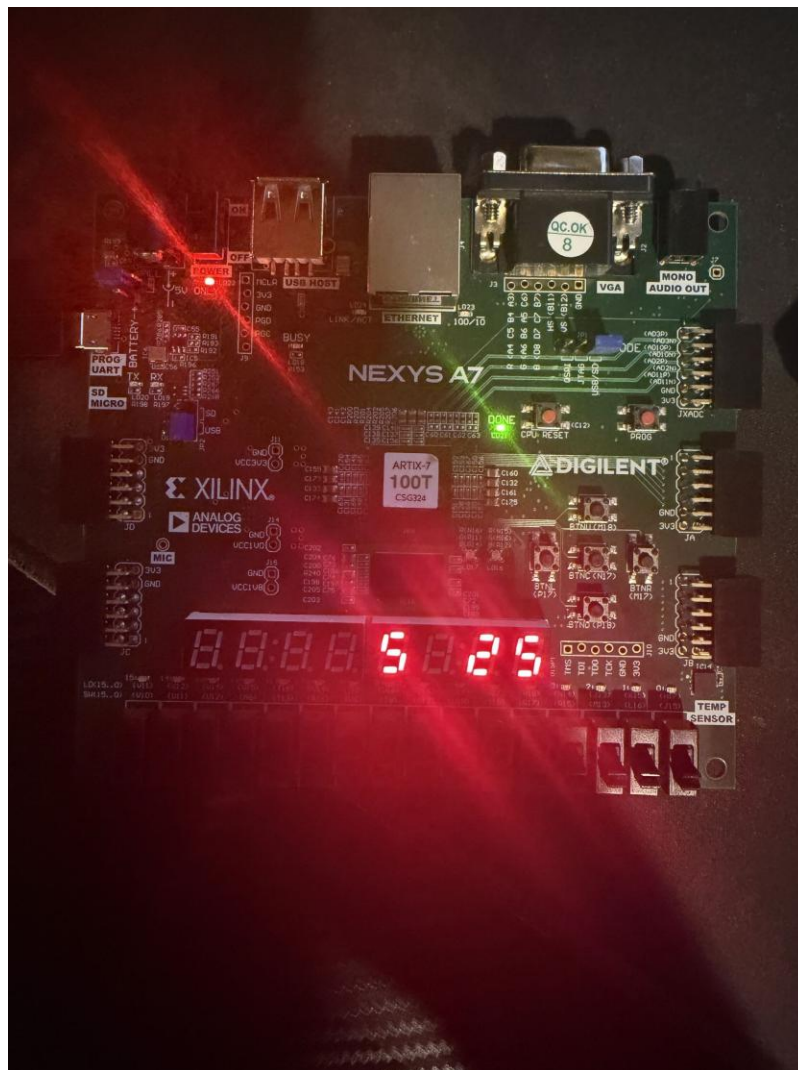


Figure 7: Squaring Test ( $5^2 = 25$ )

**Summary of Test Results**

| Operation      | FO | SO | Expected     | Actual       | Pass/Fail |
|----------------|----|----|--------------|--------------|-----------|
| Addition       | 5  | 4  | 9            | 9            | Pass      |
| Multiplication | 5  | 3  | 15           | 15           | Pass      |
| Subtraction    | 1  | 4  | -3 (LED lit) | -3 (LED Lit) | Pass      |
| Division       | 4  | 2  | 2            | 2            | Pass      |
| Squaring       | 5  | -  | 25           | 25           | Pass      |

All five operations produced correct results on the physical hardware. The seven-segment display showed bright digits with no dimming, which was an issue that we ran into before. This showed that the code is running in a stable and controlled manner. The negative LED asserted properly during subtraction with a negative result and remained off for all other operations.

## **Conclusion**

This project successfully demonstrated the design and implementation of a simple calculator on the Nexys FPGA board. The calculator takes in inputs from 1 to 5 through push buttons and performs addition, multiplication, subtraction, division, and squaring using slide switches for operation selection. All operations were verified both in simulation and on the physical hardware.

By splitting the calculator into four focused sub-modules connected through a top-level module, each one could be understood and debugged independently. The debouncer handles only button cleaning, the FSM handles only state transitions, the ALU handles only arithmetic, and the display driver handles only multiplexing. This separation of concerns made it straightforward to locate and fix issues during development.

The finite state machine provided a clear and structured way to manage the flow of the calculator, from accepting inputs to performing the operation and displaying the result. Using an FSM made it straightforward to handle the different paths through the calculator, like in the squaring bypass where only one operand is needed.

The seven-segment multiplexing driver was one of the more important parts of the design. The Nexys A7 shares a single cathode bus across all eight digits, so only one digit can be physically lit at a time. By cycling through the active digits, the driver makes all digits appear simultaneously. An earlier version of the design used separate display outputs for each digit, which caused all displays to be dim because multiple anodes were being driven at the same time.

Two's complement subtraction was implemented explicitly in the ALU by inverting the bits of the second operand, adding one, and adding the result to the first operand. Checking the MSB of the eight-bit result determines the sign. If negative, the two's complement of the result gives the absolute value for display. This way it follows the subtraction method taught in class and the one we were instructed to use.

Overall, this project reinforced concepts including modular design, finite state machine design, Verilog coding for synthesis, FPGA pin constraints, seven-segment display multiplexing, button debouncing, and two's complement arithmetic. The calculator works correctly for all five of the operations and gives the results through the seven segment display and negative LEDS